

Protobuf Contract and Maven/Nexus Publishing Flow

A concise guide to defining, packaging, publishing, and consuming a shared gRPC contract

Core idea

The `.proto` file defines the communication contract. Maven generates and packages the Java classes, while Nexus distributes the resulting JAR to other projects.

Learning reference

1. Define the Contract

The `.proto` file is the single source of truth for communication between services. It describes the service methods, request messages, response messages, field names, and data types.

```
syntax = "proto3";

package banking.account.v1;

service AccountService {
  rpc GetAccountBalance(GetAccountBalanceRequest)
    returns (GetAccountBalanceResponse);
}

message GetAccountBalanceRequest {
  string account_id = 1;
}

message GetAccountBalanceResponse {
  string account_id = 1;
  int64 balance_in_paisa = 2;
  string currency = 3;
}
```

Important: Both the client and server follow this contract. They do not need to manually agree on request and response formats.

2. Generate Java Classes

Maven uses the Protobuf plugin to read the `.proto` file and generate Java source code.

Input	Processing	Output
account.proto	Maven Protobuf Plugin	Generated Java classes

The generated code commonly includes request classes, response classes, gRPC service classes, client stubs, and server base classes.

3. Package the Generated Classes

Maven compiles the generated Java code and packages the compiled classes into a JAR file.

```
account.proto
  ↓
Maven Protobuf Plugin
  ↓
Generated Java classes
  ↓
banking-contracts-1.0.0.jar
```

The artifact is identified using Maven coordinates:

Group ID	com.learning.banking
Artifact ID	banking-contracts

Version	1.0.0
Complete coordinates	com.learning.banking:banking-contracts:1.0.0

4. Publish the JAR to Nexus

The contract project's `pom.xml` uses `distributionManagement` to tell Maven where the artifact should be published.

```
<distributionManagement>
  <repository>
    <id>nexus-releases</id>
    <name>Nexus Releases</name>
    <url>
      http://localhost:8081/repository/maven-releases/
    </url>
  </repository>
</distributionManagement>
```

The artifact is published with:

```
mvn deploy
```

After deployment, the JAR is available in the configured Nexus repository for other Maven projects to download.

5. Consume the Contract

A consuming project first declares the Nexus repository so Maven knows where to search for the artifact.

```
<repositories>
  <repository>
    <id>nexus-releases</id>
    <url>
      http://localhost:8081/repository/maven-releases/
    </url>
  </repository>
</repositories>
```

Then it adds the contract JAR as a Maven dependency:

```
<dependency>
  <groupId>com.learning.banking</groupId>
  <artifactId>banking-contracts</artifactId>
  <version>1.0.0</version>
</dependency>
```

The consuming project can now use the generated Java classes without copying the original `.proto` file or generating the classes again.

6. Important Maven Distinction

Configuration	Purpose
<code>distributionManagement</code>	Tells Maven where to publish or deploy an artifact.

repositories	Tells Maven where to search for and download dependencies.
--------------	--

7. Complete Flow

account.proto
↓
Maven Protobuf Plugin
↓
Generated Java classes
↓
Packaged into a JAR
↓
Published to Nexus
↓
Downloaded as a Maven dependency
↓
Used by client and server projects

Summary: The .proto file defines the communication contract, Maven generates and packages the Java classes, and Nexus distributes the resulting JAR to other projects.